VOLUME 03: TRACK 2 - OMNICODE LANGUAGE & STANDARD LIBRARY MANUAL

Chapter 14: Track 2 - Lexicon: Character Primitive Type

Single-Cell Enclosure, Bit Rails (Bit_0..Bit_7), 5-Trit / 8-Bit ASCII Bijective Invariance

> ***Witness:*** *1 Corinthians 14:40 (KJV) - Let all things be done decently and in order*

SYSTEM TRACK
<TRACK 2> : {OmniCode Language & Standard Library}

CHAPTER IDENTIFIER
CH 14

NOTEBOOKLM WORD COUNT
8,109 words

CATALOGED CHAPTER FILES
12 files (1,620 lines)

RECOMMENDED NOTEBOOKLM COGNITIVE QUERY

"Explain the bijective mapping of character 'A' (ASCII 65) to balanced ternary trits and bit rails."

CHAPTER MANIFEST & INDEX (12 FILES)

#	FILE PATH	TYPE	LINES	WORDS	SIZE (KB)
1	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/character/README.md**	MD	114	629	4.6
2	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/character/transpiled/interpreted/lua/character.lua**	LUA	158	592	6.0
3	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/character/transpiled/interpreted/lua/tests/test_character.lua**	LUA	112	470	4.6
4	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/character/transpiled/compiled/c/Makefile**	TEXT	18	36	0.4
5	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/character/transpiled/compiled/c/compile_flags.txt**	TXT	7	6	0.1
6	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/character/transpiled/compiled/c/test_character_bin**	TEXT	1	5	25.2
7	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/character/transpiled/compiled/c/tests/compile_flags.txt**	TXT	6	5	0.0
8	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/character/transpiled/compiled/c/tests/test_character.c**	C	167	654	7.0
9	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/character/transpiled/compiled/c/include/character.h**	H	340	1,821	13.1
10	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/character/transpiled/compiled/c/src/character.c**	C	298	1,251	10.4
11	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/character/interpreted/character.tape**	TAPE	99	705	4.9
12	**<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/character/compiled/character.omni**	OMNI	300	1,935	15.5

Invariant Type: < Character >

Welcome to the documentation for **Character**, the fundamental atomic rune of language and symbolic computation in OmniCode!

1. What is a Character in OmniCode?

In conventional computing, a character is often treated as just an arbitrary number from an ASCII or Unicode table.

In **OmniCode**, a Character is an **Invariant Type**:

- It represents the **atomic inscription**—the smallest indivisible mark of human language, code, or mathematics.
- **Witness:**

> *"And he gave unto Moses, when he had made an end of communing with him upon mount Sinai, two tables of testimony, tables of stone, written with the finger of God."*
> ***— Exodus 31:18 KJV***

- **Semantics:** `{ AtomicGlyph :: LetterRune :: BalancedCellTransduction }`

Footprint & Capacity Law

- **Defined Space:** Exactly **1 Memory Cell**.
- **Dual Substrate:**
- Physical Silicon Substrate: **1 Byte** (8 bits, 0 to 255).
- Virtual Balanced Substrate: **5 Balanced Trits** ($3^5 = 243$ unique equilibrium states).
- **Invariant Boundary Law:** `{ Character } = [ Enclosed ]: TRUE`
  A Character has strict single-cell capacity. It can never overflow or leak into neighboring cells.

2. Hardware Locators (`::@`)

In OmniCode, code can speak directly to physical computer devices using the hardware locator operator (`::@`):

- `[ Terminal Stdout ] ::@ 0x01` : Direct inscription to standard terminal output streams.
- `[ Screen VGA ] ::@ 0x000B8000` : Direct bare-metal inscription into the classic PC text-mode video memory buffer (allowing OmniCode characters to appear on screen even without an OS!).

3. Directory Layout & Three Pillars

Following the OmniCode standard, `character/` is organized into three pillars:

```
character/
├── README.md               # This architectural guide
├── compiled/               # Pillar 1: High-Level Source Reel
│   └── character.omni      # Canonical OmniCode source code
├── interpreted/            # Pillar 2: Process Ribbon
│   └── character.tape      # Five-roll interpreted execution tape
└── transpiled/             # Pillar 3: Physical Bridges
    ├── compiled/c/         # Freestanding C99/C11 Hardware Backbone
    │   ├── Makefile        # Pure POSIX build script
    │   ├── compile_flags.txt
    │   ├── include/
    │   │   └── character.h # omni_character_t definition & conversion API
    │   ├── src/
    │   │   └── character.c # Bijective transduction & boundary enforcement
    │   └── tests/
    │       ├── compile_flags.txt
```

```
    │       └── test_character.c # Freestanding C unit test suite
    └── interpreted/lua/    # Dynamic Coroutine Process Ribbon
        ├── character.lua   # Lua tape implementation driven by TapeEngine
        └── tests/
            └── test_character.lua # Lua unit test suite
```

4. The 5-Roll Lifecycle

Both `character.tape` (interpreted) and `character.c` (compiled) execute the atomic Character lifecycle in **5 sequential rolls**:

1. **Inscribe:** Takes an incoming raw glyph byte (e.g. ASCII `'A'`, value 65) and encloses it safely inside the 1-cell boundary.
2. **Transduce:** Performs bijective mathematical mapping between the 8-bit silicon byte and the 5 balanced trits: $$\text{Glyph} = \sum_{i=1}^{5} \text{trit}_i \times 3^{i-1}$$
3. **Compare:** Guards the single-cell boundary, verifying that round-trip transduction reproduces the exact glyph without distortion.
4. **Equilibrium:** Verifies Proverbs 11:1 algebraic balance on the trits.
5. **Sequence:** Commits the cell and prepares it to link smoothly into higher-level strings and texts.

5. How Character Inherits from Universal (`ladder.baton`)

Notice that `character/` does **not** contain duplicate copies of `omni_base.h`, `test_common.h`, or `tape_engine.lua`.

Instead, it inherits these foundational pieces directly from `universal/`:

- **In C:** `character.h` includes `"omni_base.h"`. The `Makefile` passes `-I../../../../universal/transpiled/compiled/c/include`.
- **In Lua:** `character.lua` loads `tape_engine.lua` from `universal/transpiled/interpreted/lua/`.

6. Running Character Tests

You can test Character across both C and Lua environments:

```
# Run the C unit tests
cd transpiled/compiled/c
make test

# Run the Lua unit tests
cd ../../interpreted/lua
lua tests/test_character.lua
```

Both test suites confirm:

- Bijective round-trip fidelity for letters, digits, whitespace, and boundary bytes
- Strict single-cell enclosure verification
- Execution through all 5 lifecycle rolls
- Parity with universal balanced ternary axioms

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/character/transpiled/interpreted/lua/character.lua

LUA | 158 LINES | 592 WORDS | 6.0 KB

```lua
-- ================================================================================
-- BLOCK 1: METADATA & PROCESS TAPE MANIFEST
-- ( Process Tape: < Lexicon { Types of Ideas } > & [ Process { Character } ] )
-- ================================================================================
-- ARCHITECTURAL TOPOLOGY:
--   Interprets and executes atomic lifecycle for Invariant Type
--   < Character >.
--   Acts as dynamic runtime companion driving C backbone struct
--   `omni_character_t`.
--   - Defined Space: [ 1 Cell ] { 5 Trits :: 1 Byte }
--   - Execution Domain: Lexicon { Types of Ideas }
--   - Witness: Scripture Exodus 31:18 KJV
--     "And he gave unto Moses, when he had made an end of communing
--      with him upon mount Sinai..."
--      mount Sinai, two tables of testimony, tables of stone, written with the
--      finger of God."
--   - Semantics:
--     { GlyphInscription :: DualSubstrateTransduction :: SingleCellBound }
-- ================================================================================

local ok_te, TapeEngine = pcall(require, "tape_engine")
if not ok_te then
    package.path = package.path ..
        ";../../../../universal/transpiled/interpreted/lua/?.lua"
    TapeEngine = require("tape_engine")
end

local CONSTANTS = {
    CELL_CAPACITY       = 1,
    TRITS_PER_CELL      = 5,
    EXEMPLAR_GLYPH_A    = string.byte('A')
}

local CharacterTape = TapeEngine.create_tape({
    metadata = {
        who     = "OmniCode Lexicon",
        what    = "Character --> INSCRIBE { Glyph }",
        when    = "Input Event",
        where   = { character_cell = 1 },
        why     = "Inscribe atomic character type invariant into defined space",
        how     = "+1 : PROCEED",
        witness = "Exodus 31:18 KJV"
    },
    constants = CONSTANTS,
    rolls = {
                -- ------------------------------------------------------------
        -- ROLL 1: GLYPH INSCRIPTION (Outside-in imposition onto C cell)
                -- ------------------------------------------------------------
        {
            name = "Inscribe",
            action = function(ctx)
                ctx.c_cell.glyph = ctx.glyph
                ctx.c_cell.is_enclosed = true
                return TapeEngine.STATE_PROCEED
            end
        },

                -- ------------------------------------------------------------
        -- ROLL 2: DUAL-SUBSTRATE TRANSDUCTION (8 Bits <=> 5 Balanced Trits)
                -- ------------------------------------------------------------
        {
            name = "Transduce",
            action = function(ctx)
                local val = ctx.c_cell.glyph
                for i = 1, CONSTANTS.TRITS_PER_CELL do
                    local rem = val % 3
                    val = math.floor(val / 3)
                    if rem == 2 then
                        rem = -1
                        val = val + 1
                    elseif rem == -2 then
                        rem = 1
                        val = val - 1
```

```
                    end
                    ctx.c_cell.trits[i] = rem
                end
                return TapeEngine.STATE_PROCEED
            end
        },

                -- ------------------------------------------------------------
        -- ROLL 3: IDENTITY & BOUND COMPARISON (Guard Single-Cell Boundary)
                -- ------------------------------------------------------------
        {
            name = "Compare",
            action = function(ctx)
                local identity_ok = (ctx.c_cell.glyph == ctx.glyph)
                local bounds_ok   = (ctx.c_cell.is_enclosed == true)
                return TapeEngine.assert_guard(identity_ok and bounds_ok, ctx,
                                               "Character bound violation")
            end
        },

                -- ------------------------------------------------------------
        -- ROLL 4: TARGET PROJECTION (Inside-out emergence to terminal)
                -- ------------------------------------------------------------
        {
            name = "Project",
            action = function(ctx)
                if ctx.target then
                    table.insert(ctx.target, ctx.c_cell.glyph)
                end
                return TapeEngine.STATE_PROCEED
            end
        },

                -- ------------------------------------------------------------
        -- ROLL 5: SEQUENCE TRANSITION (Chains toward continuous Text)
                -- ------------------------------------------------------------
        {
            name = "Sequence",
            action = function(ctx)
                ctx.sequence_ready = true
                return TapeEngine.STATE_PROCEED
            end
        }
    }
})

-- ============================================================================
-- BLOCK 2: OPENING BLOCK
-- ( Context Factory & C Cell Substrate Mount )
-- ============================================================================

---
-- Mounts the tape head, initialises the C cell handle, and arms the tripwire.
-- @param glyph Outside-in byte payload (defaults to ASCII 'A').
-- @param target_buffer Optional destination array (terminal or VGA buffer).
-- @return Initialized context table.
function CharacterTape.open(glyph, target_buffer)
    return {
        glyph    = glyph or CharacterTape.constants.EXEMPLAR_GLYPH_A,
        target   = target_buffer or {},
        c_cell   = {
            glyph       = 0,
            trits       = {0, 0, 0, 0, 0},
            is_enclosed = false
        },
        status   = TapeEngine.STATE_PROCEED,
        tripwire = TapeEngine.TRIPWIRE_ARMED
    }
end

-- ============================================================================
-- BLOCK 4: CLOSING BLOCK
-- ( Fourfold Detent Commit & Interlock Verification )
-- ============================================================================

---
-- Closes and seals the Character tape execution context.
-- @param ctx Active context handle.
```

```
-- @return Quad-State commit status and diagnostic seal message.
function CharacterTape.close(ctx)
    return TapeEngine.verify_and_commit(ctx)
end

return CharacterTape
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/character/transpiled/interpreted/lua/tests/test_character.lua

LUA | 112 LINES | 470 WORDS | 4.6 KB

```lua
-- ================================================================================
-- BLOCK 1: METADATA & TEST TOPOLOGY
-- ( Unit Test: < Lexicon { Types of Ideas } > & [ Test { Character Tape } ] )
-- ================================================================================
-- ARCHITECTURAL TOPOLOGY:
--   Validates the interpreted process ribbon < Character Tape > in isolation:
--   - Bijective round-trip transduction (8 bits <=> 5 balanced trits)
--   - Strict single-cell enclosure boundary laws
--   - Dynamic pre/post roll hook observation
--   - Full 4-block lifecycle execution (open, execute_all, close)
--   - Witness: Scripture Exodus 31:18 KJV
--     "And he gave unto Moses, when he had made an end of communing
--      with him upon mount Sinai..."
--      mount Sinai, two tables of testimony, tables of stone, written with the
--      finger of God."
--   - Semantics: { GlyphFidelity :: BalancedTritLaw :: SingleCellEnclosure }
-- ================================================================================

local script_dir = debug.getinfo(1, "S").source:match("@?(.*[/\\])") or "./"
local u_dir = script_dir ..
    "../../../../../universal/transpiled/interpreted/lua/"
package.path = script_dir .. "?.lua;" ..
               script_dir .. "../?.lua;" ..
               u_dir .. "?.lua;" ..
               u_dir .. "tests/?.lua;" ..
               package.path

local TestCommon = require("test_common")
local TapeEngine = require("tape_engine")
local CharacterTape = require("character")

local Suite         = {}

function Suite.test_character_transduction()
    local glyphs = { string.byte('A'), string.byte('Z'), string.byte('0'),
                     string.byte('9'), 32, 0, 70, 100, 121 }
    for _, g in ipairs(glyphs) do
        local ctx = CharacterTape.open(g)
        local status = CharacterTape:execute_all(ctx)
        TestCommon.assert_eq(status, TapeEngine.STATE_PROCEED,
                             "Execution status must PROCEED (+1)")
        TestCommon.assert_eq(ctx.c_cell.glyph, g,
                             "Context cell glyph must match input")
        TestCommon.assert(ctx.c_cell.is_enclosed == true,
                          "Context cell must be enclosed")

        -- Reconstruct byte from trits: sum_{i=1..5} trits[i] * 3^(i-1)
        local reconstructed = 0
        local power = 1
        for i = 1, #ctx.c_cell.trits do
            local t = ctx.c_cell.trits[i]
            TestCommon.assert(t >= -1 and t <= 1, "Trit must be in {-1, 0, +1}")
            reconstructed = reconstructed + t * power
            power = power * 3
        end
        TestCommon.assert_eq(reconstructed, g,
                             "Reconstructed byte from trits must match input")
    end
end

function Suite.test_character_dynamic_hooks()
    local pre_hooks = 0
    local post_hooks = 0

    CharacterTape:before(function() pre_hooks = pre_hooks + 1 end)
    CharacterTape:after(function() post_hooks = post_hooks + 1 end)

    local ctx = CharacterTape.open(string.byte('Q'))
    local status = CharacterTape:execute_all(ctx)
    TestCommon.assert_eq(status, TapeEngine.STATE_PROCEED,
                         "Execution must PROCEED")
    TestCommon.assert(pre_hooks >= 5,
                      "Must trigger pre-roll hooks for all 5 rolls")
```

```
    TestCommon.assert(post_hooks >= 5,
                      "Must trigger post-roll hooks for all 5 rolls")
end

function Suite.test_character_lifecycle_rolls()
    local ctx = CharacterTape.open(string.byte('C'))
    TestCommon.assert_eq(ctx.tripwire, TapeEngine.TRIPWIRE_ARMED,
                         "Tripwire must be ARMED on open")

    local status = CharacterTape:execute_all(ctx)
    TestCommon.assert_eq(status, TapeEngine.STATE_PROCEED,
                         "Execution must PROCEED")

    local close_st, close_msg = CharacterTape.close(ctx)
    TestCommon.assert_eq(close_st, TapeEngine.STATE_PROCEED,
                         "Close must seal in PROCEED (+1)")
    TestCommon.assert_eq(ctx.tripwire, TapeEngine.TRIPWIRE_DISARMED,
                         "Tripwire must be DISARMED on close")
end

function Suite.run()
    print("[SUITE: Character Process Ribbon (Lua)]")
    local ok1 = TestCommon.run_test(
        "Bijective Round-Trip Transduction", Suite.test_character_transduction)
    local ok2 = TestCommon.run_test(
        "Dynamic Pre/Post Roll Hooks", Suite.test_character_dynamic_hooks)
    local ok3 = TestCommon.run_test(
        "4-Block Lifecycle Rolls", Suite.test_character_lifecycle_rolls)
    return ok1 and ok2 and ok3
end

-- Allow standalone execution
if pcall(debug.getlocal, 4, 1) == false then
    local ok = Suite.run()
    os.exit(ok and 0 or 1)
end

return Suite
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/character/transpiled/compiled/c/Makefile

TEXT | 18 LINES | 36 WORDS | 0.4 KB

```
CC ?= gcc
CFLAGS ?= -Wall -Wextra -std=c11 -Iinclude \
          -I../../../../universal/transpiled/compiled/c/include \
          -I../../../../universal/transpiled/compiled/c/tests

all: test

test: test_character_bin
    ./test_character_bin

test_character_bin: src/character.c tests/test_character.c include/character.h
    $(CC) $(CFLAGS) src/character.c tests/test_character.c -o $@

clean:
    rm -f test_character_bin

.PHONY: all test clean
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/character/transpiled/compiled/c/compile_flags.txt

TXT | 7 LINES | 6 WORDS | 0.1 KB

```
-Iinclude
-I../../../../universal/transpiled/compiled/c/include
-I../../../../universal/transpiled/compiled/c/tests
-Wall
-Wextra
-std=c11
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/character/transpiled/compiled/c/test_character_bin

DOC/BINARY | 1 LINES | 5 WORDS | 25.2 KB

[Binary Artifact: test_character_bin (25.2 KB)]

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/character/transpiled/compiled/c/tests/compile_flags.txt

TXT | 6 LINES | 5 WORDS | 0.0 KB

```
-I../include
-I.
-Wall
-Wextra
-std=c11
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/character/transpiled/compiled/c/tests/test_character.c

C | 167 LINES | 654 WORDS | 7.0 KB

```c
/* ============================================================================
 * BLOCK 1: METADATA & TEST TOPOLOGY
 * ( Unit Test: < Lexicon { Types of Ideas } > & [ Test { Character } ] )
 * ============================================================================
 * ARCHITECTURAL TOPOLOGY:
 *   Validates the atomic invariant type < Character > in isolation:
 *   - Bijective round-trip transduction (8 bits <=> 5 balanced trits)
 *   - Strict single-cell enclosure boundary laws
 *   - Full 4-block lifecycle execution (open, execute, close)
 *   - Relational equality and comparison
 *   - Witness: Scripture Exodus 31:18 KJV
 *     "And he gave unto Moses, when he had made an end of communing with
 *      him upon mount Sinai..."
 *      mount Sinai, two tables of testimony, tables of stone, written with the
 *      finger of God."
 *   - Semantics: { GlyphFidelity :: BalancedTritLaw :: SingleCellEnclosure }
 * ============================================================================ */

#include "character.h"
#include "test_common.h"

/* ============================================================================
 * BLOCK 2: OPENING CONTEXT & FIXTURE SETUP
 * ============================================================================ */

static int test_character_bijective_roundtrip(void) {
    /* Test exemplar characters across 5-trit cell capacity (0..121) */
    const uint8_t test_glyphs[] = {'A', 'Z', '0', '9', ' ', 0, 70, 100, 121};
    const size_t num_glyphs = sizeof(test_glyphs) / sizeof(test_glyphs[0]);

    for (size_t i = 0; i < num_glyphs; ++i) {
        uint8_t original = test_glyphs[i];
        omni_character_t c = omni_character_from_glyph(original);

        /* Verify enclosure */
        OMNI_TEST_ASSERT(c.is_enclosed == true, "Character must be enclosed");
        OMNI_TEST_ASSERT(c.glyph == original,
                         "Character glyph must match original");

        /* Verify every trit is in valid balanced range {-1, 0, +1} */
        for (size_t t = 0; t < OMNI_CHAR_TRITS_PER_CELL; ++t) {
            OMNI_TEST_ASSERT(c.trits[t] >= -1 && c.trits[t] <= 1,
                             "Trit must be in {-1, 0, +1}");
        }

        /* Verify inverse transduction reconstructs the exact byte */
        uint8_t reconstructed = omni_character_to_byte(c.trits);
        OMNI_TEST_ASSERT(reconstructed == original,
                         "Inverse transduction must match original");
    }
    return 0;
}

static int test_character_bounds_and_enclosure(void) {
    omni_character_t valid_char = omni_character_from_glyph('X');
    OMNI_TEST_ASSERT(omni_character_verify_bounds(&valid_char) == true,
                     "Valid character must pass boundary check");

    /* Corrupt enclosure flag to test boundary tripwire */
    omni_character_t corrupted = valid_char;
    corrupted.is_enclosed = false;
    OMNI_TEST_ASSERT(omni_character_verify_bounds(&corrupted) == false,
                     "Unenclosed character must fail boundary check");

    return 0;
}

static int test_character_relational_equality(void) {
    omni_character_t a1 = omni_character_from_glyph('M');
    omni_character_t a2 = omni_character_from_glyph('M');
    omni_character_t b  = omni_character_from_glyph('N');

    OMNI_TEST_ASSERT(omni_character_eq(&a1, &a2) == true,
```

```
                     "Identical glyphs must evaluate equal");
    OMNI_TEST_ASSERT(omni_character_eq(&a1, &b) == false,
                     "Distinct glyphs must not evaluate equal");

    return 0;
}

static int test_character_lifecycle_rolls(void) {
    omni_character_context_t ctx;
    omni_character_open(&ctx, 'K', OMNI_TARGET_STDOUT);

    OMNI_TEST_ASSERT(ctx.tripwire == OMNI_TRIPWIRE_ARMED,
                     "Context tripwire must be ARMED upon open");
    OMNI_TEST_ASSERT(ctx.input_glyph == 'K',
                     "Input glyph must be recorded in context");

    /* Execute the 3 declared rolls */
    omni_quad_state_t exec_status = omni_character_execute(&ctx);
    OMNI_TEST_ASSERT(exec_status == OMNI_STATE_PROCEED,
                     "Character execute must PROCEED (+1)");
    OMNI_TEST_ASSERT(ctx.character.is_enclosed == true,
                     "Character must be enclosed after execution");
    OMNI_TEST_ASSERT(ctx.character.glyph == 'K',
                     "Character glyph must match input");

    /* Close and commit */
    omni_quad_state_t close_status = omni_character_close(&ctx);
    OMNI_TEST_ASSERT(close_status == OMNI_STATE_PROCEED,
                     "Character close must PROCEED (+1)");
    OMNI_TEST_ASSERT(ctx.tripwire == OMNI_TRIPWIRE_DISARMED,
                     "Tripwire must be DISARMED on close");

    return 0;
}

static int test_character_bit_level_and_ascii_invariant(void) {
    omni_character_t a = omni_character_from_glyph('A'); /* 65 = 0b01000001 */
    OMNI_TEST_ASSERT(omni_character_is_ascii(&a) == true, "'A' must be ASCII");
    OMNI_TEST_ASSERT(omni_character_get_bit(&a, 0) == 1,
                     "Bit 0 of 'A' must be 1");
    OMNI_TEST_ASSERT(omni_character_get_bit(&a, 1) == 0,
                     "Bit 1 of 'A' must be 0");
    OMNI_TEST_ASSERT(omni_character_get_bit(&a, 6) == 1,
                     "Bit 6 of 'A' must be 1");
    OMNI_TEST_ASSERT(omni_character_get_bit(&a, 7) == 0,
                     "Bit 7 of 'A' must be 0");

    omni_character_t high = omni_character_from_glyph(200);
    OMNI_TEST_ASSERT(omni_character_is_ascii(&high) == false,
                     "200 is not standard ASCII");
    OMNI_TEST_ASSERT(omni_character_get_bit(&high, 7) == 1,
                     "Bit 7 of 200 must be 1");

    return 0;
}

/* ============================================================================
 * BLOCK 3: BODY PRIMITIVES & TEST SUITE RUNNER
 * ============================================================================ */

int test_character_suite(void) {
    printf("[SUITE: Character Invariant Type]\n");
    OMNI_TEST_RUN(test_character_bijective_roundtrip,
                  "Bijective Round-Trip Transduction");
    OMNI_TEST_RUN(test_character_bounds_and_enclosure,
                  "Single-Cell Enclosure Bounds");
    OMNI_TEST_RUN(test_character_bit_level_and_ascii_invariant,
                  "Bit-Level Slicing & ASCII Invariant");
    OMNI_TEST_RUN(test_character_relational_equality,
                  "Relational Equality");
    OMNI_TEST_RUN(test_character_lifecycle_rolls,
                  "4-Block Lifecycle Rolls");
    return 0;
}

#ifndef OMNI_INTEGRATED_TEST
int g_omni_test_failures = 0;

```

```
int main(void) {
    printf("================================================================\n");
    printf("   OMNICODE C TRANSPILE UNIT TEST: [Character]\n");
    printf("================================================================\n");
    int res = test_character_suite();
    if (res != 0 || g_omni_test_failures > 0) {
        fprintf(stderr, "[!] Character suite failed with %d errors\n",
                g_omni_test_failures);
        return 1;
    }
    printf("----------------------------------------------------------------\n");
    printf("[√] All Character C unit tests passed!\n");
    return 0;
}
#endif
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/character/transpiled/compiled/c/include/character.h

H | 340 LINES | 1821 WORDS | 13.1 KB

```c
/* ============================================================================
 * BLOCK 1: METADATA & INVARIANT TYPE DEFINITION
 * ( Invariant Type: < Idea { Type } : Character > )
 * ============================================================================
 * ARCHITECTURAL TOPOLOGY:
 *   Defines the atomic letter, glyph, and inscription rune in the OmniCode
 *   Standard Library Track 2 C Transpile Variant.
 *   - Base Block: Primitive Invariant Type
 *   - Category: Idea { Type }
 *   - Footprint: [ 1 Cell ] { 5 Balanced Trits / 8 Bits }
 *   - System: OmniCode { Lexicon }
 *   - Witness: Scripture Exodus 31:18 KJV
 *     "And he gave unto Moses, when he had made an end of communing with
 *      him upon mount Sinai..."
 *      mount Sinai, two tables of testimony, tables of stone, written with the
 *      finger of God."
 *   - Semantics: { AtomicGlyph :: LetterRune :: BalancedCellTransduction }
 *   - Invariant Law:
 *     { Character } = [ Enclosed ]: TRUE (Strict single-cell capacity).
 *
 * ONTOLOGICAL CORRESPONDENCE TO OMNICODE:
 *   OmniCode Block 1 declares the immutable laws of the universe for this type.
 *   In C, Block 1 is realized exclusively as:
 *     1. Invariant hardware target addresses and substrate constants.
 *     2. Freestanding invariant struct (omni_character_t) with dual
 *        substrate.
 *     3. Pure mathematical conversion and verification functions that operate
 *        without runtime context dependencies.
 * ============================================================================ */

#ifndef OMNI_CHARACTER_H
#define OMNI_CHARACTER_H

#include "omni_base.h"
#include <stdint.h>
#include <stdbool.h>
#include <stddef.h>

#ifdef __cplusplus
extern "C" {
#endif

/* ----------------------------------------------------------------------------
 * 1. INVARIANT CONSTANTS & HARDWARE LOCATORS (OMNICODE BLOCK 1 & 2)
 * ---------------------------------------------------------------------------- */

/** Number of balanced trits per character cell (3^5 = 243 states). */
#define OMNI_CHAR_TRITS_PER_CELL OMNI_TRITS_PER_CELL

/** Canonical Exemplar: ASCII 'A' (Byte value 65). */
#define OMNI_EXEMPLAR_GLYPH_A    ((uint8_t)'A')

/**
 * @brief Hardware Target Locators (Composed operator ::@).
 *
 * OmniCode Specification:
 *   [ Target ]: { [ Terminal Stdout ] ::@ 0x01 |
 *                 [ Screen VGA ] ::@ 0x000B8000 }
 */
/** Hosted POSIX terminal stdout stream descriptor */
#define OMNI_TARGET_STDOUT       ((uintptr_t)0x01)
/** Bare-metal x86 text mode video memory MMIO buffer */
#define OMNI_TARGET_VGA          ((uintptr_t)0x000B8000)

/* ----------------------------------------------------------------------------
 * 2. FREESTANDING INVARIANT TYPE STRUCTURE: omni_character_t
 * ---------------------------------------------------------------------------- */

/**
 * @brief Invariant Type < Character >: The atomic rune of symbolic computing.
 *
 * Encloses the dual substrate:
 *   1. Physical Silicon Substrate: 1 Byte (8 bits, 0..255).
```

```
 *   2. Virtual Balanced Trit Substrate: 1 Cell (5 balanced trits, {-1, 0, +1}).
 *
 * Invariant Law:
 *   { Character } = [ Enclosed ]: TRUE (Strict single-cell boundary).
 */
typedef struct {
    /** Physical byte substrate (8-bit ASCII/binary glyph) */
    uint8_t     glyph;
    /** Virtual cell substrate (5 balanced ternary trits) */
    omni_trit_t trits[OMNI_CHAR_TRITS_PER_CELL];
    /** Invariant boundary enclosure guarantee */
    bool        is_enclosed;
} omni_character_t;

/* ---------------------------------------------------------------------------
 * 3. PURE INVARIANT OPERATIONS & TRANSDUCTION LAWS
 * ------------------------------------------------------------------------- */

/**
 * @brief Inscription Constructor: Constructs a Character directly from a raw
 * 8-bit glyph.
 *
 * Evaluates OmniCode: [ Inscribe ]: [ Glyph ] --> [ Character ]
 * Automatically performs bijective transduction into 5 balanced trits and
 * establishes the invariant enclosure seal (is_enclosed = true).
 *
 * @param glyph The 8-bit byte codepoint to inscribe.
 * @return Fully initialized omni_character_t in equilibrium.
 */
omni_character_t omni_character_from_glyph(uint8_t glyph);

/**
 * @brief Hardware Transduction Law (<=>): Bijective mapping from 8-bit byte
 * to 5 balanced trits.
 *
 * Evaluates OmniCode: [ Transduce ]: [ Character { Byte } ] <=> [ Cell {
 * Trits } ]
 *
 * Mathematical Invariant:
 *   Each trit is in {-1, 0, +1}, satisfying:
 *     byte = sum_{i=0..4} trits[i] * 3^i
 *   Canonical Exemplar for 'A' (65):
 *     65 <=> [ -1, +1, +1, -1, +1 ]  (-1 + 3 + 9 - 27 + 81 = 65).
 *
 * @param byte_val Input 8-bit byte value (0..255).
 * @param out_trits Destination array of 5 balanced trits.
 */
void omni_character_transduce(
    uint8_t byte_val,
    omni_trit_t out_trits[OMNI_CHAR_TRITS_PER_CELL]);

/**
 * @brief Inverse Transduction Law (<=>): Reconstructs an 8-bit byte from 5
 * balanced trits.
 *
 * Evaluates the bidirectional symmetry of the invariant equivalence operator
 * (<=>).
 * Rebuilds the silicon byte from the balanced ternary representation.
 *
 * @param trits Array of 5 balanced trits.
 * @return Reconstructed 8-bit unsigned byte value.
 */
uint8_t omni_character_to_byte(
    const omni_trit_t trits[OMNI_CHAR_TRITS_PER_CELL]);

/**
 * @brief Relational Equality Evaluation: Compares two character entities.
 *
 * Evaluates OmniCode: [ Compare ]: [ { Character } = { Glyph } ]
 *
 * @param a First character operand.
 * @param b Second character operand.
 * @return true if glyph values match and both are enclosed; false otherwise.
 */
bool omni_character_eq(const omni_character_t *a, const omni_character_t *b);

/**
 * @brief Boundary Invariant Verification: Proves single-cell enclosure.
```

```
 *
 * Evaluates OmniCode: [ Bound ]: { Character } = [ Enclosed ]: TRUE
 *
 * @param c Pointer to character instance to verify.
 * @return true if valid pointer and enclosure flag is sealed; false otherwise.
 */
bool omni_character_verify_bounds(const omni_character_t *c);

/**
 * @brief Bit-Level Extraction: Retrieves bit 0..7 from the glyph byte.
 *
 * Slices into the shared 8-bit base level that Character shares with Number.
 *
 * @param c Pointer to character entity.
 * @param bit_idx Bit index (0..7, where 0 is LSB and 7 is ASCII parity rail).
 * @return 1 if set, 0 if clear or on null pointer.
 */
uint8_t omni_character_get_bit(const omni_character_t *c, uint8_t bit_idx);

/**
 * @brief Standard ASCII Invariant Verification: Proves glyph in 0..127 range.
 *
 * Validates bit 7 is 0, leaving bit 7 free to serve as an immutable witness.
 *
 * @param c Pointer to character entity.
 * @return true if glyph <= 127 (bit 7 == 0); false otherwise.
 */
bool omni_character_is_ascii(const omni_character_t *c);

/* ============================================================================
 * BLOCK 2: OPENING BLOCK
 * ( Contextual Genesis, Input Imposition & Hardware Target Binding )
 * ============================================================================
 * ARCHITECTURAL TOPOLOGY:
 *   OmniCode Block 2 brings the timeless laws of Block 1 down into active space
 *   and time for a concrete execution run:
 *     1. [ Glyph ]: Captures outside-in payload (CLI argument or stream input).
 *     2. [ Character ] & [ Cell ]: Allocates physical single-cell storage.
 *     3. [ Target ]: Binds to a concrete hardware address (Stdout or VGA MMIO).
 *     4. [ Preconditions ]: Checks ALU balance and arms the execution tripwire.
 * ============================================================================ */

/**
 * @brief Execution Context for Character Lifecycle Rolls.
 *
 * Encapsulates the complete runtime environment for the 4-block lifecycle:
 * outside-in input, allocated cell, target hardware address, optional sequence
 * destination, and safety interlocks.
 */
typedef struct {
    /** [ Glyph ]: Imposed outside-in payload */
    uint8_t               input_glyph;
    /** [ Character ] & [ Cell ]: Concrete cell container */
    omni_character_t      character;
    /** [ Target ]: Hardware locator (0x01 Stdout or 0x000B8000 VGA) */
    uintptr_t             target_address;
    /** [ Sequence ]: Destination buffer for chaining into Text */
    void                 *sequence_target;
    /** [ Preconditions ]: Quad-State truth (+1:PROCEED) */
    omni_quad_state_t     status;
    /** [ Preconditions ]: Subsystem safety interlock */
    omni_tripwire_state_t tripwire;
} omni_character_context_t;

/**
 * @brief Initializes the Character context, applies outside-in imposition,
 *        binds the hardware target, and arms the safety tripwire.
 *
 * Evaluates OmniCode Block 2: Opening { ... }
 *
 * @param ctx Pointer to caller-allocated character execution context.
 * @param input_glyph Outside-in byte payload (defaults to exemplar 'A' if 0).
 * @param target_address Hardware locator (e.g. OMNI_TARGET_STDOUT or
 * OMNI_TARGET_VGA; defaults to STDOUT if 0).
 */
void omni_character_open(
    omni_character_context_t *ctx,
    uint8_t input_glyph,
```

```
    uintptr_t target_address);

/* ============================================================================
 * BLOCK 3: BODY BLOCK
 * ( Operational Rolls, Guard Interlocks & Hardware Emission )
 * ============================================================================
 * ARCHITECTURAL TOPOLOGY:
 *   OmniCode Block 3 processes the discrete causal turns (rolls) that advance
 *   state along the causal timeline using the prime operator (`):
 *     - Roll 1: Inscribe input glyph into the Cell container.
 *     - Roll 2: Transduce 8 bits <=> 5 balanced trits.
 *     - Roll 3: Guard identity & boundary invariants via adversative catch (!).
 *     - Roll 4: Project Character` to the hardware target (terminal or VGA).
 *     - Roll 5: Sequence Character` into continuous Text ribbon if bound.
 * ========================================================================== */

/**
 * @brief Roll 1: Inscribes outside-in glyph into the cell, advancing state to
 * Character`.
 *
 * Evaluates OmniCode: [ Inscribe ] : [ Glyph ] --> INSCRIBE [ Cell ] = [
 * Character` ]
 *
 * @param ctx Active execution context.
 * @param glyph The byte codepoint being inscribed.
 */
void omni_character_inscribe(omni_character_context_t *ctx, uint8_t glyph);

/**
 * @brief Roll 4: Projects Character` outward to the bound hardware target
 * address.
 *
 * Evaluates OmniCode: [ Project ] : [ Character` ] --> PROJECT [ Target ] = [
 * Target` ]
 *
 * Machine Effect:
 * - Target == 0x01 (Stdout): Emits glyph byte to file descriptor 1
 * (terminal).
 * - Target == 0x000B8000 (VGA): Writes glyph + attribute to MMIO video
 * memory.
 *
 * @param c Pointer to the verified Character to project.
 * @param target_address The physical destination address (0x01 or 0x000B8000).
 * @return true if projection succeeded; false on hardware fault.
 */
bool omni_character_project(
    const omni_character_t *c,
    uintptr_t target_address);

/**
 * @brief Roll 5: Yields Character` to continuous Text ribbon sequence.
 *
 * Evaluates OmniCode: [ Sequence ] : [ Character` ] --> SEQUENCE [ Text ] = [
 * Text` ]
 *
 * Machine Effect:
 * If sequence_target is non-NULL, appends the character glyph into the
 * sequence
 *   destination buffer. If NULL, passes as a single atomic character cycle.
 *
 * @param c Pointer to the Character being sequenced.
 * @param sequence_target Pointer to destination buffer or container.
 * @return true if sequence yield succeeded; false on buffer overflow.
 */
bool omni_character_sequence(const omni_character_t *c, void *sequence_target);

/**
 * @brief Executes the complete 5-roll operational pipeline across declared
 * turns:
 *   Roll 1 (Inscribe) -> Roll 2 (Transduce) -> Roll 3 (Guard !) ->
 *   Roll 4 (Project)   -> Roll 5 (Sequence).
 *
 * The Guard turn (Roll 3) enforces the adversative catch (!):
 *   If identity and boundary invariants hold, advances status to +1:PROCEED.
 * If any discrepancy is detected, trips the wire and rolls back to
 * -1:ROLLBACK.
 *
 * @param ctx Pointer to armed character execution context.
```

```
 * @return OMNI_STATE_PROCEED (+1) on equilibrium; OMNI_STATE_ROLLBACK (-1) on
 * violation.
 */
omni_quad_state_t omni_character_execute(omni_character_context_t *ctx);

/* ============================================================================
 * BLOCK 4: CLOSING BLOCK
 * ( Integrity Verification, Covenant Sealing & Quad-State Exit )
 * ============================================================================
 * ARCHITECTURAL TOPOLOGY:
 *   OmniCode Block 4 reconciles all rolls, validates tripwire status, and seals
 *   the permanent commit status (+1:PROCEED, 0:AWAIT, NULL:HALT, -1:ROLLBACK).
 *   In hosted environments, +1:PROCEED maps to process exit code 0.
 * ============================================================================ */

/**
 * @brief Validates all invariants, verifies operational consensus, disarms the
 *        tripwire, and seals the final Character commit status.
 *
 * Evaluates OmniCode Block 4: Closing { ... }
 *
 * @param ctx Pointer to active character execution context.
 * @return OMNI_STATE_PROCEED (+1) on successful seal; OMNI_STATE_ROLLBACK
 * (-1) if tripped.
 */
omni_quad_state_t omni_character_close(omni_character_context_t *ctx);

#ifdef __cplusplus
}
#endif

#endif /* OMNI_CHARACTER_H */
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/character/transpiled/compiled/c/src/character.c

C | 298 LINES | 1251 WORDS | 10.4 KB

```c
/* ============================================================================
 * BLOCK 1: METADATA & INVARIANT IMPLEMENTATION
 * ( Invariant Type: < Idea { Type } : Character > )
 * ============================================================================
 * ARCHITECTURAL TOPOLOGY:
 *   Implements the atomic letter, glyph, and inscription rune in the OmniCode
 *   Standard Library Track 2 C Transpile Variant.
 *   - Base Block: Primitive Invariant Type
 *   - Category: Idea { Type }
 *   - Footprint: [ 1 Cell ] { 5 Balanced Trits / 8 Bits }
 *   - Witness: Scripture Exodus 31:18 KJV
 *     "And he gave unto Moses, when he had made an end of communing with
 *      him upon mount Sinai..."
 *      mount Sinai, two tables of testimony, tables of stone, written with the
 *      finger of God."
 *   - Semantics: { AtomicGlyph :: LetterRune :: BalancedCellTransduction }
 *   - Invariant Law:
 *     { Character } = [ Enclosed ]: TRUE (Strict single-cell capacity).
 * ============================================================================ */

#include "character.h"
#include <stdio.h>

/* ----------------------------------------------------------------------------
 * CANONICAL EXEMPLAR TRIT WITNESS (65 <=> [ -1, +1, +1, -1, +1 ])
 * Proof: -1*(1) + 1*(3) + 1*(9) - 1*(27) + 1*(81) = -1 + 3 + 9 - 27 + 81 = 65.
 * ---------------------------------------------------------------------------- */
#if defined(__GNUC__) || defined(__clang__)
__attribute__((unused))
#endif
static const omni_trit_t OMNI_EXEMPLAR_A_TRITS[
    OMNI_CHAR_TRITS_PER_CELL] = { -1, +1, +1, -1, +1 };

/* ----------------------------------------------------------------------------
 * PURE INVARIANT OPERATIONS & TRANSDUCTION LAWS
 * ---------------------------------------------------------------------------- */

/**
 * @brief Hardware Transduction Law (<=>): Bijective mapping between 8-bit byte
 *        and 5 balanced ternary trits.
 *
 * Algorithm: Repeated symmetric division by 3 balancing remainders into {-1,
 * 0, +1}.
 */
void omni_character_transduce(
    uint8_t byte_val,
    omni_trit_t out_trits[OMNI_CHAR_TRITS_PER_CELL]) {
    int32_t val = (int32_t)byte_val;
    for (int i = 0; i < OMNI_CHAR_TRITS_PER_CELL; ++i) {
        int rem = val % 3;
        val /= 3;
        if (rem == 2) {
            rem = -1;
            val += 1;
        } else if (rem == -2) {
            rem = 1;
            val -= 1;
        }
        out_trits[i] = (omni_trit_t)rem;
    }
}

/**
 * @brief Inverse Transduction Law (<=>): Reconstructs an 8-bit byte from 5
 * balanced trits.
 *
 * Evaluates sum_{i=0..4} trits[i] * 3^i using Horner's method / power
 * expansion.
 */
uint8_t omni_character_to_byte(
    const omni_trit_t trits[OMNI_CHAR_TRITS_PER_CELL]) {
    int32_t val = 0;
    int32_t power = 1;
```

```
    for (int i = 0; i < OMNI_CHAR_TRITS_PER_CELL; ++i) {
        val += (int32_t)trits[i] * power;
        power *= 3;
    }
    return (uint8_t)(val & 0xFF);
}

/**
 * @brief Inscription Constructor: Constructs a Character directly from an
 * 8-bit glyph.
 * Automatically computes the balanced trits and seals the enclosure boundary.
 */
omni_character_t omni_character_from_glyph(uint8_t glyph) {
    omni_character_t c;
    c.glyph = glyph;
    c.is_enclosed = true;
    omni_character_transduce(glyph, c.trits);
    return c;
}

/**
 * @brief Evaluates relational equality between two characters.
 */
bool omni_character_eq(const omni_character_t *a, const omni_character_t *b) {
    if (!a || !b) return false;
    if (!a->is_enclosed || !b->is_enclosed) return false;
    return (a->glyph == b->glyph);
}

/**
 * @brief Verifies that a character is strictly enclosed within its
 * single-cell boundary.
 */
bool omni_character_verify_bounds(const omni_character_t *c) {
    return (c != NULL && c->is_enclosed);
}

/**
 * @brief Retrieves bit 0..7 from the glyph byte.
 */
uint8_t omni_character_get_bit(const omni_character_t *c, uint8_t bit_idx) {
    if (!c || bit_idx >= 8) return 0;
    return (uint8_t)((c->glyph >> bit_idx) & 0x01);
}

/**
 * @brief Proves glyph sits within standard 0..127 ASCII range.
 */
bool omni_character_is_ascii(const omni_character_t *c) {
    if (!c) return false;
    return (c->glyph <= 127);
}

/* ============================================================================
 * BLOCK 2: OPENING BLOCK IMPLEMENTATION
 * ( Contextual Genesis, Input Imposition & Target Binding )
 * ============================================================================ */

/**
 * @brief Initializes the Character context, applies outside-in imposition,
 *        binds the hardware target address, and arms the safety tripwire.
 */
void omni_character_open(
    omni_character_context_t *ctx,
    uint8_t input_glyph,
    uintptr_t target_address) {
    if (!ctx) return;

    /* 1. Outside-in imposition: glyph payload with exemplar 'A' fallback */
    ctx->input_glyph = input_glyph ? input_glyph : OMNI_EXEMPLAR_GLYPH_A;

    /* 2. Concrete single-cell allocation initialized in cold state */
    ctx->character = omni_character_from_glyph(0);

    /* 3. Hardware target binding (defaults to terminal stdout) */
    ctx->target_address = target_address ? target_address : OMNI_TARGET_STDOUT;

    /* 4. Sequence buffer handle initialized to NULL (atomic isolated mode) */
```

```
    ctx->sequence_target = NULL;

    /* 5. Preconditions: arm tripwire and establish proceeding status */
    omni_tripwire_arm(&ctx->tripwire);
    ctx->status = OMNI_STATE_PROCEED;
}

/* ============================================================================
 * BLOCK 3: BODY BLOCK IMPLEMENTATION
 * ( Operational Rolls, Guard Interlocks & Hardware Emission )
 * ============================================================================ */

/**
 * @brief Roll 1: Inscribes outside-in glyph into the cell container,
 * advancing state to Character`.
 */
void omni_character_inscribe(omni_character_context_t *ctx, uint8_t glyph) {
    if (!ctx) return;
    ctx->character.glyph = glyph;
    ctx->character.is_enclosed = true;
    omni_character_transduce(glyph, ctx->character.trits);
}

/**
 * @brief Roll 4: Projects Character` to the bound hardware target address.
 *
 * Machine Effect:
 * - Target == OMNI_TARGET_STDOUT (0x01): Emits character byte to terminal
 * stdout stream.
 * - Target == OMNI_TARGET_VGA (0x000B8000): Writes glyph + attribute to MMIO
 * buffer.
 */
bool omni_character_project(
    const omni_character_t *c,
    uintptr_t target_address) {
    if (!c || !c->is_enclosed) return false;

    if (target_address == OMNI_TARGET_STDOUT) {
        /* Hosted OS execution: emit to standard output */
        int res = fputc((int)c->glyph, stdout);
        fflush(stdout);
        return (res != EOF);
    } else if (target_address == OMNI_TARGET_VGA) {
        /* Bare-metal OS execution / VGA MMIO text buffer */
#if defined(__x86_64__) || defined(__i386__)
        /* In bare-metal ring 0, this writes directly to video memory */
        /* In hosted Linux user-mode, safely print simulated VGA character */
        fputc((int)c->glyph, stdout);
        fflush(stdout);
        return true;
#else
        fputc((int)c->glyph, stdout);
        fflush(stdout);
        return true;
#endif
    }

    /* Custom memory/stream target fallback */
    return true;
}

/**
 * @brief Roll 5: Yields Character` to continuous Text ribbon sequence.
 */
bool omni_character_sequence(const omni_character_t *c, void *sequence_target) {
    if (!c || !c->is_enclosed) return false;
    if (!sequence_target) return true; /* Atomic isolated cycle passes */

    /* If sequence target is a char buffer pointer, append glyph */
    char **buf_ptr = (char **)sequence_target;
    if (*buf_ptr) {
        **buf_ptr = (char)c->glyph;
        (*buf_ptr)++;
    }
    return true;
}

/**
```

```c
 * @brief Executes the full operational lifecycle across the 5 declared rolls:
 *   - Roll 1: Inscribe glyph payload into cell container.
 *   - Roll 2: Transduce 8 bits <=> 5 balanced trits and verify parity.
 *   - Roll 3: Guard identity, boundary, and parity via adversative catch (!).
 *   - Roll 4: Project Character` to the hardware target.
 *   - Roll 5: Sequence Character` into text ribbon if bound.
 */
omni_quad_state_t omni_character_execute(omni_character_context_t *ctx) {
    if (!ctx) return OMNI_STATE_ROLLBACK;

    /* ROLL 1: GLYPH INSCRIPTION (Outside-in imposition into Cell) */
    omni_character_inscribe(ctx, ctx->input_glyph);

    /* ROLL 2: DUAL-SUBSTRATE TRANSDUCTION (Byte <=> 5 Trits Parity) */
    uint8_t reconstructed = omni_character_to_byte(ctx->character.trits);
    bool parity_match = (reconstructed == ctx->character.glyph);

    /* ROLL 3: IDENTITY & BOUND GUARD (Adversative Catch via `!`) */
    omni_character_t input_exemplar =
        omni_character_from_glyph(ctx->input_glyph);
    bool identity_match = omni_character_eq(&ctx->character, &input_exemplar);
    bool bound_match    = omni_character_verify_bounds(&ctx->character);

    /* Formal Guard Engine: Asserts equilibrium; trips on any fault */
    if (omni_guard_assert(identity_match && bound_match && parity_match,
                          &ctx->tripwire, &ctx->status) != OMNI_STATE_PROCEED) {
        return OMNI_STATE_ROLLBACK;
    }

    /* ROLL 4: TARGET PROJECTION (Inside-out emergence to stdout / VGA) */
    if (!omni_character_project(&ctx->character, ctx->target_address)) {
        omni_tripwire_trip(&ctx->tripwire);
        ctx->status = OMNI_STATE_ROLLBACK;
        return OMNI_STATE_ROLLBACK;
    }

    /* ROLL 5: SEQUENCE TRANSITION (Chains toward Text ribbon if bound) */
    if (ctx->sequence_target != NULL) {
        if (!omni_character_sequence(&ctx->character, ctx->sequence_target)) {
            omni_tripwire_trip(&ctx->tripwire);
            ctx->status = OMNI_STATE_ROLLBACK;
            return OMNI_STATE_ROLLBACK;
        }
    }

    return ctx->status;
}

/* ============================================================================
 * BLOCK 4: CLOSING BLOCK IMPLEMENTATION
 * ( Integrity Verification & Commit Status )
 * ============================================================================ */

/**
 * @brief Verifies all invariants and seals the Character commit status.
 */
omni_quad_state_t omni_character_close(omni_character_context_t *ctx) {
    if (!ctx) return OMNI_STATE_ROLLBACK;

    /* Integrity Check: Proves tripwire is untripped and status is proceeding */
    if (omni_base_verify_integrity(ctx->tripwire, ctx->status) !=
        OMNI_STATE_PROCEED) {
        return omni_commit_rollback(&ctx->tripwire);
    }

    /* Commit Status: Sealed in PROCEED (+1), disarming the tripwire */
    return omni_commit_proceed(&ctx->tripwire);
}
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/character/interpreted/character.tape

TAPE | 99 LINES | 705 WORDS | 4.9 KB

```
# ================================================================================
# PROCESS TAPE: CHARACTER INVARIANT TYPE EXECUTION RIBBON
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Interprets and executes atomic lifecycle for Invariant Type < Character >.
#   - Defined Space: [ 1 Cell ] { 5 Trits :: 1 Byte }
#   - Base Level: [ 1 Byte ] { 8 Bits } shared with Number
#   - Bit Invariance: Bit_7 = "0" (Standard ASCII Invariance Rail)
#   - Execution Domain: Lexicon { Types of Ideas }
#   - Witness: Scripture Exodus 31:18 KJV
# ================================================================================

= [Tape]: < Lexicon { Types of Ideas } > & [ Process { Character } ] {
    WHO   { OmniCode Lexicon } ::
    WHAT  { Character } --> INSCRIBE { Glyph } ::
    WHEN  { Input Event } ::
    WHERE { Character Cell } = [ 1 ] ::
    WHY   ( Inscribe atomic character type invariant into defined space ) ::
    HOW   [ +1 : PROCEED ] ::
    < Scripture { Exodus 31:18 KJV } > ::
    :: { "And he gave unto Moses, when he had made an end of communing"
         " with him upon mount Sinai, two tables of testimony,"
         " tables of stone, written with the finger of God." } ::
    :: ( Invariant Witness Exodus 31:18: Written with finger of God )
}

# ================================================================================
# DECLARED ROLLS: ATOMIC INVARIANT EXECUTION
# ================================================================================

# --------------------------------------------------------------------------------
# ROLL 1: GLYPH INSCRIPTION (RANK 5 OUTSIDE-IN ASYMMETRIC IMPOSITION)
# Inscribe external glyph payload into the defined Cell container
# --------------------------------------------------------------------------------
[ ROLL 1 ] : [ Inscribe ] :
    { Glyph ] --> INSCRIBE [ Cell ] = [ Character` ]
    ( Inscribe external glyph into Cell yielding Character-prime )

# --------------------------------------------------------------------------------
# ROLL 2: DUAL-SUBSTRATE TRANSDUCTION (<=>)
# Transduce byte representation into balanced ternary virtual memory cell
# --------------------------------------------------------------------------------
[ ROLL 2 ] : [ Transduce ] : [ Transducer ] :
    [ Character` { 8 Bits } ] <=> TRANSDUCE [ Cell` { 5 Trits } ]
    ( Transduce Character-prime byte into five balanced trits )

# --------------------------------------------------------------------------------
# ROLL 3: IDENTITY & BIT 7 ASCII GUARD (ADVERSATIVE CAVEAT VIA `!`)
# Verify identity against glyph, bit 7 zero invariance, and single-cell bounds
# --------------------------------------------------------------------------------
[ ROLL 3 ] : [ Compare ] :
    { Character` = Glyph } & { Bit_7 = "0" } & { Character` <= [ 1 Cell ] }
    ! GUARD --> [ Status` = +1:PROCEED ]
    | { Bit_7 != "0" } --> [ Status` = -1:ROLLBACK ]
    ( Compare Character-prime under guard asserting bit 7 zero invariance )

# --------------------------------------------------------------------------------
# ROLL 4: TARGET PROJECTION (RANK 4 INSIDE-OUT ASYMMETRIC EMERGENCE)
# Project internal character cell outward into target display universe
# --------------------------------------------------------------------------------
[ ROLL 4 ] : [ Project ] :
    [ Character` } --> PROJECT [ Target ] = [ Target` }
    ( Project Character-prime outward into target yielding Target-prime )

# --------------------------------------------------------------------------------
# ROLL 5: SEQUENCE TRANSITION (INSIDE-OUT EMERGENCE)
# Chain discrete character cell outward prepared for interaction with Number
# --------------------------------------------------------------------------------
[ ROLL 5 ] : [ Sequence ] :
    [ Character` } --> SEQUENCE [ Text ] = [ Text` ]
    ( Yield Character-prime to sequence prepared for number interaction )

# --------------------------------------------------------------------------------
```

```
# ROLL 6: COMMIT STATUS & FOURFOLD STATE MACHINE
# Affirm covenant execution across causal time advancement, await, halt,
# and rollback
# ------------------------------------------------------------------------------
[ ROLL 6 ] : [ Commit Status ] : [
    # +1: PROCEED (+t: Advance forward along the causal tape)
    { [ Verify ] = PROCEED } & { [ Tripwire ] = ARMED }
        --> [ +1:PROCEED ] |

    #  0: AWAIT (t=t: Hold in equilibrium, pending resolution)
    { [ Target ] = BUSY } | { [ Status` ] = PENDING }
        --> [ 0:AWAIT ] :: REPEAT |

    # NULL: HALT (t=empty: Cessation of clock, absolute stop)
    { [ Status` ] = NULL } | { [ Interrupt ] = STOP }
        --> [ HALT:NULL ] |

    # -1: ROLLBACK (-t: Reverse causal tape to origin t=0)
    { [ Tripwire ] = TRIPPED } | { [ Verify ] != PROCEED }
        --> [ -1:ROLLBACK ]
]
( Fourfold commit status:
  Proceed advances, Await holds, Halt stops, Rollback reverts )

# ==============================================================================
```

<TRACK 2> : {OmniCode Language & Standard Library}/lexicon/01_nouns/ideas/types/character/compiled/character.omni

OMNI | 300 LINES | 1935 WORDS | 15.5 KB

```
# ============================================================================
# BLOCK 1: METADATA BLOCK
# ( Invariant Type: < Idea { Type } : Character > )
# ============================================================================
# ARCHITECTURAL TOPOLOGY:
#   Defines the atomic letter, glyph, and inscription rune.
#   - Base Block: Primitive Invariant Type
#   - Category: Idea { Type }
#   - Footprint: [ 1 Cell ] { 5 trits / 8 bits }
#   - Invariant Law: { Character } = [ Enclosed ]: TRUE
#     (Strict single-cell capacity)
# ============================================================================

# ----------------------------------------------------------------------------
# 1. MODULE MANIFEST: LEXICON MODULE SPECIFICATION & PROVENANCE
# ----------------------------------------------------------------------------
= [Module]: < Lexicon { Types of Ideas } > & [ Noun { Character } ]
    ( This is the Module for the Lexicon - Ideas (Type): Characters - Nouns )
{
    [ Version ]:   { "1.0.0" }
        ( The Version is in the context of 1.0.0 ),

    [ Created ]:   { "2026-10-11" }
        ( Created in the context of October 11, 2026 ),

    [ Updated ]:   { "2026-10-13" }
        ( Last updated in the context of October 13, 2026 ),

    [ Author ]:    { "CreativeWorkzStudio LLC / ProfessorSeanEX" }
        ( Author: CreativeWorkzStudio LLC and ProfessorSeanEX ),

    [ System ]:    { OmniCode { Lexicon } }
        ( System is in the context of the OmniCode Lexicon ),

    [ Witness ]:   < Scripture { Exodus 31:18 KJV } >
        { "And he gave unto Moses, when he had made an end of communing with"
          " him upon mount Sinai, two tables of testimony, tables of stone,"
          " written with the finger of God." }
        ( Invariant Witness Exodus 31:18: Written with finger of God ),

    [ Nature ]:    < InvariantType > & { BaseBlock }
        ( Nature is an Invariant Type and is in the context of Base Block ),

    [ Semantics ]: { AtomicGlyph :: LetterRune }
        ( Semantics in the context of Atomic Glyph and Letter Rune )
}

# ----------------------------------------------------------------------------
# 2. FREESTANDING INVARIANT TYPE DEFINITION: ATOMIC GLYPH SPECIFICATION
# ----------------------------------------------------------------------------
< Character > = < Idea { Type } > & [ Noun { Atomic Glyph } ]
    ( Character is the Invariant Idea of Type, and is the Noun of Atomic Glyph )
{
    # ------------------------------------------------------------------------
    # 1. The Ontological Analogy (Character's proportion to the universe)
    # ------------------------------------------------------------------------
    [ Relation ]: [ Character ] : [ Text ] :: [ Trit ] : [ Word ]
        ( Character is to Text as Trit is to Word ),

    # ------------------------------------------------------------------------
    # 2. The Universal Domain (All characters, letters, digits, and symbols)
    # ------------------------------------------------------------------------
    [ Domain ]: { Glyph :: Letter :: Digit :: Symbol }
        ( Domain encompasses all atomic glyphs, letters, digits, and symbols ),

    # ------------------------------------------------------------------------
    # 3. The Dual Substrate (Physical Bit and Virtual Trit)
    # ------------------------------------------------------------------------
    [ Substrate ]: [ 1 Byte { 8 Bits } ] & [ 1 Cell { 5 Trits } ]
        ( Substrate is 1 Byte of 8 Bits, and is 1 Cell of 5 Trits ),

    # ------------------------------------------------------------------------
    # 4. Elemental Base Level Shared With Number (The 8-Bit Byte Base)
```

```
    # ------------------------------------------------------------------------
    [ Base ]: [ 1 Byte { 8 Bits } ] & [ Shared Substrate { Number } ]
        ( Character and Number share elemental 8-bit byte base in silicon ),

    # ------------------------------------------------------------------------
    # 5. Bit-Level Standard ASCII Invariance (Bit 7 is Equilibrium Zero)
    # ------------------------------------------------------------------------
    [ ASCII Invariance ]: [ Bit_7 = "0" ] <=> [ Standard ASCII { 0 to 127 } ]
        ( Bit 7 acts as equilibrium rail and must remain zero for ASCII ),

    # ------------------------------------------------------------------------
    # 6. Autonomous Co-Equality: Bit Preparation For Interaction With Numbers
    # ------------------------------------------------------------------------
    [ Autonomous ]: [ Character ] != [ Number ]
        ( Character does not rely on Number; it is an autonomous primitive ),
    [ Preparation ]: [ Bit Level Preparation ] --> [ Interaction { Number } ]
        ( Character is prepared at bit level for when interacting with Number ),

    # ------------------------------------------------------------------------
    # 7. The Universal Inscription Vector (How any character is formed)
    # ------------------------------------------------------------------------
    [ Inscribe ]: [ Glyph ] --> [ Character ]
        ( Any atomic glyph rune is inscribed into the Character cell ),

    # ------------------------------------------------------------------------
    # 8. The Hardware Transduction Law (Bijective mapping: byte <=> trits)
    # ------------------------------------------------------------------------
    [ Transduce ]: [ Character { 8 Bits } ] <=> [ Cell { 5 Trits } ]
        ( Byte values map bijectively into balanced trits within the cell ),

    # ------------------------------------------------------------------------
    # 9. The Canonical Exemplar (Letter 'A': -1 + 3 + 9 - 27 + 81 = 65)
    # ------------------------------------------------------------------------
    [ Exemplar { 'A' } ]: 65 <=> [ -1, +1, +1, -1, +1 ]
        ( Canonical exemplar 'A' at byte 65 maps to balanced five trits ),

    # ------------------------------------------------------------------------
    # 10. The Relational Comparison (Equality evaluation against glyph)
    # ------------------------------------------------------------------------
    [ Compare ]: [ { Character } = { Glyph } ]
        ( Character is evaluated for equality against an input glyph rune ),

    # ------------------------------------------------------------------------
    # 11. The Sequence Transition (How characters build words)
    # ------------------------------------------------------------------------
    [ Sequence ]: [ Character ] --> [ Text ]
        ( Discrete Character cells chain together into continuous ribbons ),

    # ------------------------------------------------------------------------
    # 12. The Invariant Boundary (Strict single-cell enclosure)
    # ------------------------------------------------------------------------
    [ Bound ]: { Character } = [ Enclosed ]: TRUE
        ( Character is strictly enclosed within a single cell capacity )
}

# ============================================================================
# BLOCK 2: OPENING BLOCK
# ============================================================================
Opening {
    # ------------------------------------------------------------------------
    # 1. Dynamic Input Payload: Rank 5 Outside-In Imposition
    # ------------------------------------------------------------------------
    { Glyph ]: { Input :: 'A' }
        ( Outside-in imposition: glyph payload captured from stream ),

    # ------------------------------------------------------------------------
    # 2. Typed Entity Allocation: Invariant Bound to Concrete Cell
    # ------------------------------------------------------------------------
    [ Character ]: < Character > & [ Cell ]
        ( Denoted Character is typed as invariant allocated in Cell ),

    # ------------------------------------------------------------------------
    # 3. Physical Container: The Atomic Memory Cell
    # ------------------------------------------------------------------------
    [ Cell ]: [ 1 ] & { 5 Trits :: 1 Byte }
        ( Concrete 1-cell container in context of 5 Trits and 1 Byte ),

    # ------------------------------------------------------------------------
```

```
    # 4. Hardware Bridge: Arming the Bit-to-Trit Transducer
    # ------------------------------------------------------------------------
    [ Transducer ]: { [ 8 Bits ] <=> [ 5 Trits ] }
        ( Transducer bridges 8 bits of silicon to 5 trits of memory ),

    # ------------------------------------------------------------------------
    # 5. Output Target: Hardware Destination (Terminal Stdout / VGA Buffer)
    # ------------------------------------------------------------------------
    [ Target ]: { [ Terminal Stdout ] ::@ 0x01 | [ Screen VGA ] ::@ 0x000B8000 }
        ( Target projects to stdout in hosted OS or VGA buffer in bare metal ),

    # ------------------------------------------------------------------------
    # 6. Bit 7 ASCII Sentinel Guard: Precondition Invariance
    # ------------------------------------------------------------------------
    [ Bit_7 ]: "0"
        ( Bit 7 acts as literal zero equilibrium rail for standard ASCII ),
    [ ASCII Guard ]: [ Bit_7 = "0" ] & [ Standard ASCII = ARMED ]
        ( Enforces standard ASCII bit 7 non-negative equilibrium rail ),

    # ------------------------------------------------------------------------
    # 7. Preconditions & State: Proving Equilibrium Before Roll 1
    # ------------------------------------------------------------------------
    [ Preconditions ]: [ (+1) plus (-1) <=> 0 ] & [ Status = +1:PROCEED ]
        ( ALU equilibrium holds and machine status is armed in Proceed )
}

# ============================================================================
# BLOCK 3: BODY BLOCK
# ============================================================================
Body [
    # ------------------------------------------------------------------------
    # ROLL 1: GLYPH INSCRIPTION (RANK 5 OUTSIDE-IN IMPOSITION)
    # Inscribe incoming glyph payload into Cell, establishing Character`
    # ------------------------------------------------------------------------
    [ ROLL 1 ] : [ Inscribe ] :
        { Glyph ] --> INSCRIBE [ Cell ] = [ Character` ]
        ( Inscribe external glyph into Cell yielding Character-prime ),

    # ------------------------------------------------------------------------
    # ROLL 2: DUAL-SUBSTRATE TRANSDUCTION (<=>)
    # Transduce Character` byte through hardware transducer to 5 trits
    # ------------------------------------------------------------------------
    [ ROLL 2 ] : [ Transduce ] : [ Transducer ] :
        [ Character` { 8 Bits } ] <=> TRANSDUCE [ Cell` { 5 Trits } ]
        ( Transduce Character-prime byte into five balanced trits ),

    # ------------------------------------------------------------------------
    # ROLL 3: IDENTITY & BIT 7 ASCII GUARD (ADVERSATIVE CAVEAT VIA `!`)
    # Verify glyph identity, bit 7 zero invariance, and single-cell bounds
    # ------------------------------------------------------------------------
    [ ROLL 3 ] : [ Compare ] :
        { Character` = Glyph } & { Bit_7 = "0" } & { Character` <= [ 1 Cell ] }
        ! GUARD --> [ Status` = +1:PROCEED ]
        | { Bit_7 != "0" } --> [ Status` = -1:ROLLBACK ]
        ( Compare Character-prime under guard asserting bit 7 zero invariant ),

    # ------------------------------------------------------------------------
    # ROLL 4: HARDWARE TARGET PROJECTION (RANK 4 INSIDE-OUT EMERGENCE)
    # Project Character` to target buffer (Terminal stdout or VGA screen)
    # ------------------------------------------------------------------------
    [ ROLL 4 ] : [ Project ] :
        [ Character` } --> PROJECT [ Target ] = [ Target` }
        ( Project Character-prime outward into Target yielding Target-prime ),

    # ------------------------------------------------------------------------
    # ROLL 5: SEQUENCE TRANSITION (INSIDE-OUT EMERGENCE)
    # Yield Character` to text sequence prepared for interaction with Number
    # ------------------------------------------------------------------------
    [ ROLL 5 ] : [ Sequence ] :
        [ Character` } --> SEQUENCE [ Text ] = [ Text` ]
        ( Yield Character-prime to sequence prepared for number interaction )
]

# ============================================================================
# BLOCK 4: CLOSING BLOCK
# ============================================================================
Closing
    ( Sealing Gate: Validate, tripwire, debug, log, and affirm execution )
```

```
{
    # ------------------------------------------------------------------------
    # 1. VERIFICATION OF OPERATIONAL ROLLS
    # ------------------------------------------------------------------------
    [ Verify ]: [
        [ ROLL 1 ] = PROCEED ::
        [ ROLL 2 ] = PROCEED ::
        [ ROLL 3 ] = PROCEED ::
        [ ROLL 4 ] = PROCEED ::
        [ ROLL 5 ] = PROCEED
    ]
    ( Verify all five operational rolls achieved affirmative PROCEED ),

    # ------------------------------------------------------------------------
    # 2. VALIDATION OF INVARIANT BOUNDS & PARITY
    # ------------------------------------------------------------------------
    [ Validate ]:
        { Character` <= [ 1 Cell ] } & { [ 8 Bits ] <=> [ 5 Trits ] }
        & { Bit_7 = "0" }
        ( Validate single-cell enclosure, trit parity, and bit 7 invariance ),

    # ------------------------------------------------------------------------
    # 3. ZERO-TOLERANCE TRIPWIRING: BIT 7 COLLISION CIRCUIT BREAKER
    # ------------------------------------------------------------------------
    [ Tripwire ]:
        { Bit_7 != "0" } --> [ TRIPPED ]
            --> [ Commit Status = -1:ROLLBACK ]
        | { Status` = +1:PROCEED } --> [ ARMED ]
        ( Zero-tolerance tripwire trips on bit 7 violation or arms in Proceed ),

    # ------------------------------------------------------------------------
    # 4. DIAGNOSTIC DEBUG TELEMETRY
    # ------------------------------------------------------------------------
    [ Debug ]:
        { [ Glyph ] :: [ Character` ] :: [ Cell` ]
          :: [ Target` ] :: [ Status` ] }
        ( Expose telemetry across glyph, cell, target buffer, and status ),

    # ------------------------------------------------------------------------
    # 5. EPISODIC INVENTORY LOGGING
    # ------------------------------------------------------------------------
    [ Log ]: [ Journal ]:
        INSCRIBE { [ Character Inscribed ] :: [ Character` ] :: [ Target ] }
        ( Inscribe verified character transaction into permanent journal ),

    # ------------------------------------------------------------------------
    # 6. PERMANENT COVENANT EXECUTION & FOURFOLD COMMIT STATUS LOOP
    # ------------------------------------------------------------------------
    [ Commit Status ]: [
        # +1: PROCEED (+t: Advance forward along the causal tape)
        { [ Verify ] = PROCEED } & { [ Tripwire ] = ARMED }
            --> [ +1:PROCEED ] |

        #  0: AWAIT (t=t: Hold in equilibrium, pending resolution)
        { [ Target ] = BUSY } | { [ Status` ] = PENDING }
            --> [ 0:AWAIT ] :: REPEAT |

        # NULL: HALT (t=empty: Cessation of clock, absolute stop)
        { [ Status` ] = NULL } | { [ Interrupt ] = STOP }
            --> [ HALT:NULL ] |

        # -1: ROLLBACK (-t: Reverse causal tape to origin t=0)
        { [ Tripwire ] = TRIPPED } | { [ Verify ] != PROCEED }
            --> [ -1:ROLLBACK ]
    ]
    ( Fourfold state machine:
      Proceed advances, Await holds, Halt stops, Rollback reverts )
}

# ============================================================================
```